Research Paper Analysis

Online Self-Supervised Deep Learning for Intrusion Detection Systems

Technical Documentation — critical analysis, proposed improvements (SSID+) and experimental validation

Paper analysed: M. Nakıp and E. Gelenbe, “Online Self-Supervised Deep Learning for Intrusion Detection Systems,” IEEE Transactions on Information Forensics and Security, vol. 19, pp. 5668–5683, 2024. DOI: 10.1109/TIFS.2024.3402148

Team member	Roll number	Main responsibility
<Member 1>	<roll no.>	Problem, motivation, related work (§1–2)
<Member 2>	<roll no.>	System model, methodology, algorithms (§3–4)
<Member 3>	<roll no.>	Evaluation of results, critical assessment (§5–6)
<Member 4>	<roll no.>	Improvements SSID+, implementation, experiments (§7–8)

Course: Information Security — Research Paper Analysis Assignment

Colour convention used throughout: ■ amber tags mark content summarised from the original authors; ■ teal tags mark the team's own analysis, proposals and results.

Contents

Executive Summary 3

1. Research Problem, Motivation and Objectives 4

1.1 Problem . . . 4

1.2 Motivation and significance . . . 4

1.3 Objectives and claimed contributions . . . 4

1.4 Challenges involved . . . 4

2. Existing Approaches (Related Work) 4

3. System Model and Architecture 5

3.1 IDS used inside SSID: AADRNN . . . 6

3.2 Traffic metrics (inputs) . . . 6

4. Proposed Methodology and Algorithms 6

4.1 Detection (forward pass and decision) . . . 6

4.2 Learning the AADRNN . . . 6

4.3 Self-supervised packet selection . . . 7

4.4 Trust coefficient . . . 7

4.5 Phase control . . . 7

5. Experimental Setup and Key Findings of the Paper 7

Key findings reported by the authors . . . 8

6. Critical Assessment 9

6.1 Strengths . . . 9

6.2 Restrictive assumptions . . . 9

6.3 Technical drawbacks and errors found in the paper . . . 9

6.4 Gaps in the evaluation . . . 10

6.5 Unresolved issues . . . 10

7. Proposed Improvements: SSID+ 11

7.1 Further extensions (proposed, not implemented) . . . 11

8. Implementation and Experimental Validation 12

8.1 Implementation . . . 12

8.2 Experimental setup . . . 12

8.3 Results . . . 13

8.4 Discussion . . . 14

8.5 Ablation study . . . 16

8.6 Hyper-parameter sensitivity of the paper's SSID . . . 16

8.7 Threats to validity . . . 17

9. Conclusion 17

References 17

Appendix A — Reproducing the results 18

Appendix B — Individual contributions 18

Executive Summary

Nakıp and Gelenbe propose SSID, a framework that trains a deep-learning anomaly-based intrusion detection system (IDS) fully online, with no offline dataset, no human labelling and no extra generative model. The IDS labels its own training data: every packet is selected as a benign training sample with probability $\Gamma(1-y)$, where y is the IDS's estimated attack probability and $\Gamma \in [0,1]$ is a trust coefficient that SSID computes online. Γ is the product of a representativeness factor (how well the traffic learned so far matches the traffic now being observed, measured with a closed-form KL divergence) and a generalisation factor (data adequacy plus a running estimate of the training error). Trust also decides when to retrain. The model updates only when Γ drops below $\Theta = 0.95$ and no attack is in progress. With an Auto-Associative Deep Random Neural Network (AADRNN), the authors report TPR above 0.995 at an FPR of about 10^{-5} on Kitsune-Mirai after learning from only 4,161 packets, and gains of up to 18% median balanced accuracy for compromised-device identification on Bot-IoT.

Our analysis finds the idea sound and very lightweight, but we identify several weaknesses the paper does not discuss. (i) The self-labelling loop can be poisoned by an attack that ramps up slowly. (ii) The bootstrap can deadlock (Γ stays at 0 forever) if the untrained model's first output is 1. (iii) The knowledge term κ is biased upward, and eqs. (10), (28) and (38) contain typos. (iv) The exponential-traffic assumption and the cumulative representativeness measure react slowly to drift. (v) The evaluation has no variance over runs, no sensitivity analysis on the scale-dependent threshold γ, no comparison with the online KitNET detector, and no adversarial scenario.

We re-implemented SSID with AADRNN and MLP backbones in Python and propose SSID+, which makes five changes: a sliding-window Jensen–Shannon representativeness factor, a bias-corrected κ, confidence-gated pseudo-labelling, an adaptive intrusion threshold and a forgetting-factor OS-ELM. We ran three scenarios over 5 seeds. On an abrupt Mirai-like flood and on gradual benign drift, our SSID reproduction matches the paper's qualitative claims (balanced accuracy 0.995 and 0.984). Under a slow-ramp attack the paper's SSID learns the attack as normal (TPR 0.002), whereas SSID+ keeps detecting it (TPR 0.985) with the same false-positive rate.

1. Research Problem, Motivation and Objectives

■ ORIGINAL AUTHORS' CONTRIBUTION — summarised from Nakıp & Gelenbe [1]

1.1 Problem

Botnets such as Mirai compromise large numbers of cheap IoT devices and use them for Distributed Denial-of-Service (DDoS) attacks, data leakage and further spread of malware. The authors cite reports that 27.7% of global website traffic in 2021 came from malicious bots. Two tasks matter during an ongoing attack: detecting malicious traffic, which enables a reactive response, and identifying compromised devices, which enables preventive isolation.

1.2 Motivation and significance

Anomaly-based IDSs learn a model of normal traffic, so they can detect zero-day attacks. Their decisions depend on that model, though, and normal IoT traffic changes over time (devices join or leave, usage patterns shift). A model that is not updated produces more and more false alarms. Online learning is the natural fix, but the paper identifies two obstacles:

- No labelled data: enough collected and labelled traffic is rarely available for each network the IDS protects, and human labelling is slow and error-prone.
- When to update: frequent updates waste the limited compute of IoT nodes for little gain, while infrequent updates leave the model stale. The right update interval depends on the IDS's current state and on the traffic itself.

1.3 Objectives and claimed contributions

- A fully online self-supervised framework (SSID) that works with any IDS that has learnable parameters, needs no offline data collection or training, and needs no human intervention.
- A trust coefficient $\Gamma = C_{rep} \cdot C_{gen}$ that measures the trustworthiness of the IDS online.
- Self-supervised packet selection that labels training packets using only the IDS's own outputs, weighted by Γ.
- Trust-triggered updates: parameters change only when trust falls below a threshold and no attack is detected.
- Evaluation on Kitsune and Bot-IoT for two tasks, with AADRNN and MLP backbones, against offline, incremental and sequential learning and against KNN and Lasso.

1.4 Challenges involved

A system that labels its own training data has to avoid confirmation bias: a wrong decision becomes training data and makes the next wrong decision more likely. It also has to start from nothing (cold start), and it must run on devices with little memory and compute. These challenges guide our critical assessment in Section 6.

2. Existing Approaches (Related Work)

■ ORIGINAL AUTHORS' CONTRIBUTION — summarised from Nakıp & Gelenbe [1]

The paper groups prior work into three families. Botnet/DDoS detection covers SVM, MLP, DT, NB, KNN and RF classifiers, feature selection (CorrAUC), BiLSTM, N-BaIoT deep autoencoders and KitNET. Most of these are trained offline and many are supervised. Compromised-device identification covers port-scan analysis, evidence theory, federated anomaly detection (DÏoT), wavelet analysis of resource usage, and mobility-based detection. Self-supervised IDS work is summarised in the table below.

Work	Self-supervision mechanism	Needs offline data?	Extra generative / contrastive model?	Online?
Song & Kim [9]	Noised pseudo-normal data from LSTM; Inception-ResNet	Yes	Yes (LSTM generator)	No
Wang et al. [10]	BYOL + transfer learning	Yes (normal + attack)	Yes (contrastive)	No
Zhang et al. [11]	Bidirectional GAN + adversarial training	Yes	Yes (GAN)	No
Kye et al. [12]	Hierarchical AE with self-supervised signals	Yes (normal)	No	No
Caville et al. [13]	GNN (E-GraphSAGE) contrastive	Yes	Yes (contrastive)	No
Abououf et al. [15]	LSTM-AE trained in the cloud	Yes	No	Partly
Meyer et al. [16]	Federated AE self-supervision	Yes (local)	No	No
SSID [1]	IDS's own decisions weighted by trust Γ	No	No	Yes

Table 1 — Comparison of self-supervised IDS approaches (team's condensed re-statement of Table I in [1]).

■ TEAM'S OWN ANALYSIS / CONTRIBUTION

The table omits one relevant baseline: KitNET/Kitsune [17], which is itself an online, unsupervised ensemble of autoencoders, published together with the Kitsune dataset that this paper uses. KitNET also needs only a short benign "grace period" to start. That makes it the most natural competitor to SSID, yet it is not compared (see §6.4).

3. System Model and Architecture

■ ORIGINAL AUTHORS' CONTRIBUTION — summarised from Nakıp & Gelenbe [1]

SSID wraps around an existing IDS. The detection line is left unchanged. A parallel learning line reads the IDS's decisions, decides how far it trusts them, selects and labels packets, and decides when to update parameters (Fig. 1).

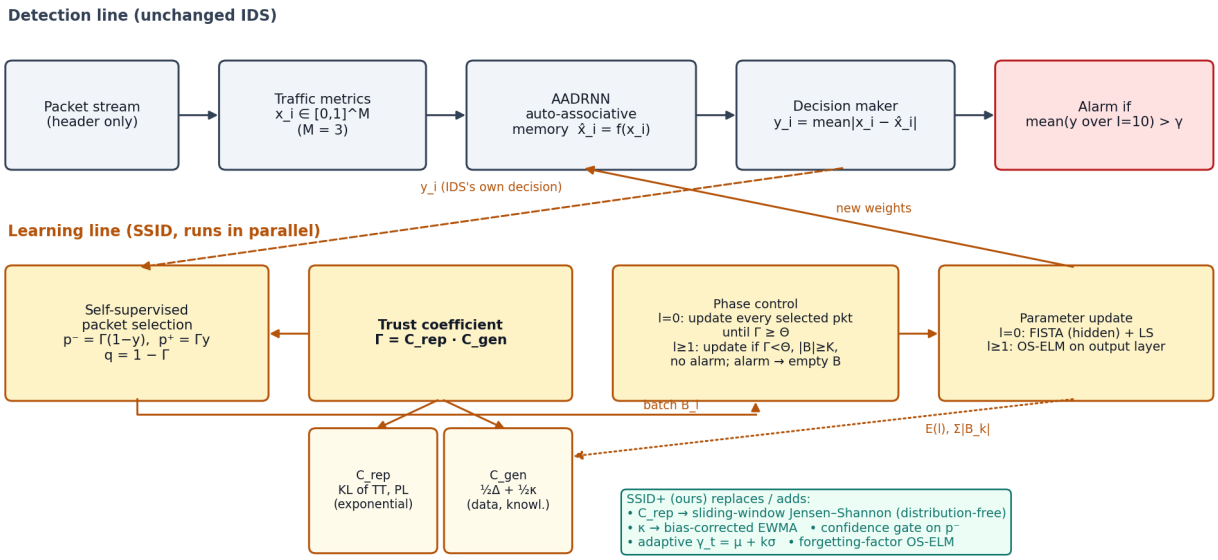


Figure 1 — SSID architecture (team's re-drawing of Figs. 2–3 of [1]). The teal box lists the changes in our SSID+ (§7).

3.1 IDS used inside SSID: AADRNN

The IDS is an auto-associative memory (AAM) $f_{aam}: [0,1]^M \to [0,1]^M$ that maps observed traffic metrics x_i to the metrics expected under benign traffic, $\hat{x}_i$. A large reconstruction deviation points to an attack. The AAM is a Deep Random Neural Network (DRNN) with H = M layers: M−1 hidden layers of M dense neuronal clusters, then M linear output neurons (Fig. 2).

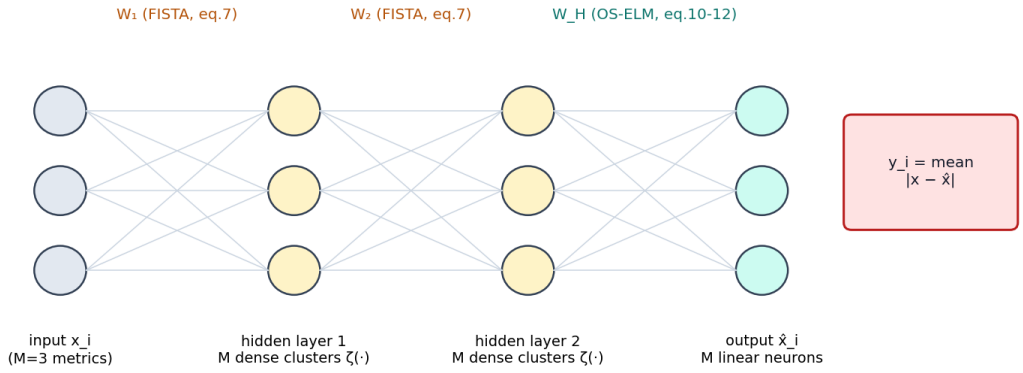


Figure 2 — AADRNN-based IDS (team's re-drawing of Fig. 1 of [1]).

3.2 Traffic metrics (inputs)

- Malicious traffic detection (M = 3): total size of the last 500 packets, average inter-transmission time of the last 500 packets, and number of packets in the last 100 s.
- Compromised device identification (M = 6, 10-s windows): average size and number of packets from a single source, maximum size and number from any single source, and total size and number of packets transmitted.

All metrics come from packet headers only. They are content-agnostic and device-agnostic, which helps privacy and keeps the computation small enough for lightweight nodes.

4. Proposed Methodology and Algorithms

■ ORIGINAL AUTHORS' CONTRIBUTION — summarised from Nakıp & Gelenbe [1]

4.1 Detection (forward pass and decision)

$$\hat{x}_{(i,1)} = \zeta([x_i, 1]\, W_1),\ \hat{x}_{(i,h)} = \zeta([\hat{x}_{(i,h-1)}, 1]\, W_h),\ \hat{x}_i = [\hat{x}_{(i,H-1)}, 1]\, W_H \quad (1)–(3)$$

$$\zeta(\Lambda) = [p(r+\lambda^+)+\lambda^-+\Lambda] \,/\, (2[\lambda^-+\Lambda]) - \sqrt{\left([p(r+\lambda^+)+\lambda^-+\Lambda] \,/\, (2[\lambda^-+\Lambda])\right)^2 - \lambda^+/(\lambda^-+\Lambda)}\ \ (4)$$

$$y_i = (1/M)\, \Sigma_m\, |x_{mi} - \hat{x}_{mi}| \text{ (traffic) } y_i = \max_m |x_{mi} - \hat{x}_{mi}| \text{ (device identification) } (5)–(6)$$

4.2 Learning the AADRNN

Initial learning (l = 0): each hidden layer is fitted with non-negative, L1-regularised least squares, solved by FISTA:

$$W_h = \text{argmin}_{W \geq 0} \,\| [\text{adj}(\zeta(\bar{X}_{h-1}\, W_R)), 1]\, W - \bar{X}_{h-1} \|^2 + \|W\|_1,\ W_h \leftarrow 0.1\, W_h \,/\, \max(\bar{X}_h)\ (7), (9)$$

Online incremental learning (l ≥ 1): only the output layer is updated, using the OS-ELM recursive least-squares rule:

$$O_l = O_{l-1} - O_{l-1} X^T (I + X\, O_{l-1}\, X^T)^{-1}\, X\, O_{l-1},\ W_H \leftarrow W_H + O_l\, X^T (\bar{X}_0 - X\, W_H),\ X = \bar{X}_{H-1}\ (10)–(12)$$

The learning error passed to SSID is the mean IDS output over the batch, $E(l) = (1/|B_l|)\, \Sigma_{i \in B_l}\, y_i$ (13).

4.3 Self-supervised packet selection

$$p^{+}_{i} = \Gamma\, y_i,\ p^{-}_{i} = \Gamma\,(1 - y_i),\ q_i = 1 - \Gamma \quad (17)\text{–}(20)$$

A packet becomes a benign training sample with probability p^{-}, a malicious sample with probability p^{+}, or is skipped with probability q. When trust is low, almost nothing is selected. The very first packet is assumed benign ($p^{-} = 1$).

4.4 Trust coefficient

$$\Gamma = C_{rep} \cdot C_{gen} \quad (21)$$

Representativeness. Inter-transmission times (TT) and packet lengths (PL) are assumed independent and exponential. For the learned set (rate λ_l) and the observed-normal set (rate λ_o), the KL divergence has a closed form:

$$D^{TT}_{KL} = \log(\lambda_o/\lambda_l) - (\lambda_o - \lambda_l)/\lambda_o,\ C_{rep} = \tfrac{1}{2}\left[\,(\lambda_l/\lambda_o)\, e^{-(\lambda_l-\lambda_o)/\lambda_o} + (\mu_l/\mu_o)\, e^{-(\mu_l-\mu_o)/\mu_o}\,\right] \quad (23),\ (32)$$

Generalisation. Generalisation is taken as "data + knowledge": $C_{gen} = \tfrac{1}{2}\Delta + \tfrac{1}{2}\kappa$ (33). Here $\Delta = 1 - \min(W / \Sigma_k |B_k|, 1)$ (34), with W the number of learnable parameters, and $\kappa = 1 - \Sigma_{k=0..l}\, (\tfrac{1}{2})^{l-k+1}\, E(k)$ (35), which can be updated recursively as $\kappa \leftarrow \tfrac{1}{2} - \tfrac{1}{2}(E(l) - \kappa)$ (36).

4.5 Phase control

- Initial learning (l = 0): the parameters update for every selected packet. The phase ends when $\Gamma \geq \Theta$ (14).
- Online learning (l ≥ 1): selected packets go into a batch B_l. The model updates when $\Gamma < \Theta$, $|B_l| \geq K$, and the mean of the last I outputs is ≤ γ (15). If that mean exceeds γ, an attack is assumed and B_l is emptied (16), so false negatives just before an alarm are not learned.

Algorithm 1 — SSID main loop (team's pseudocode of §IV in [1])

```
Γ ← 0 ; l ← 0 ; B ← ∅ ; first packet labelled benign
for each arriving packet i:
    y_i ← IDS(x_i) ;  s_i ← mean(y_{i-I+1..i})            # detection, unchanged
    if s_i ≤ γ: add (TT_i, PL_i) to observed-normal statistics
    Γ ← C_rep · C_gen
    draw u ~ U(0,1): benign if u < Γ(1−y_i); malicious if u < Γ; else skip
    if l = 0:
        if benign: update IDS with packet i ; update C_gen with E
        if Γ ≥ Θ: l ← 1
    else:
        if benign: B ← B ∪ {i}
        if s_i > γ: B ← ∅                                  # attack: discard batch
        elif Γ < Θ and |B| ≥ K:
            update IDS on B (OS-ELM) ; update C_rep, C_gen ; l ← l+1 ; B ← ∅
```

Parameter values used in [1]: $\Theta = 0.95$, I = 10, K = 100, $\gamma = 0.25$, $c_1 = c_2 = c_3 = c_4 = 0.5$.

5. Experimental Setup and Key Findings of the Paper

■ ORIGINAL AUTHORS' CONTRIBUTION — summarised from Nakıp & Gelenbe [1]

Item	Setting in [1]
Datasets	Kitsune Mirai: 764,137 packets, 107 IPs, ~7,137 s, 121,621 benign / 642,516 malicious. Kitsune SYN DoS (2.77 M packets). Bot-IoT DDoS-HTTP/TCP/UDP and DoS-HTTP (19.8 k – 19.5 M packets).
Tasks	Malicious-traffic detection (M = 3) and compromised-device identification, CDIS (M = 6, 10-s windows).
Baselines	AADRNN and MLP with offline learning (first 83,000 benign packets) and with incremental learning (retrained every 750 packets on own decisions, first 750 assumed benign); KNN and Lasso (offline); CDIS with sequential learning.
Metrics	ROC, accuracy, TPR, TNR; balanced accuracy for CDIS.

Table 2 — Experimental setup reported in [1].

Key findings reported by the authors

- SSID-AADRNN reaches TPR > 0.995 at FPR ≈ 10^{-5} on Kitsune-Mirai (ROC, Fig. 4 of [1]).
- Γ starts at 0, rises quickly, and accurately reflects how trustworthy the decisions are. The model is not updated while the attack is in progress (Fig. 5 of [1]).
- Learning fully online from only 4,161 packets, SSID is competitive with AADRNN trained offline on about 83,000 packets, and it clearly beats incremental learning on all metrics (Fig. 6 of [1]).
- SSID-MLP has slightly higher accuracy and TPR than offline MLP, which however has 100% TNR (Fig. 7 of [1]). SSID-MLP is second-best among all models (Fig. 8 of [1]).
- SSID-CDIS matches sequential CDIS on Mirai and improves it on SYN DoS. On Bot-IoT it raises median balanced accuracy by 18% (DDoS-TCP) and 11% (DDoS-UDP) and by 2% on average for DoS-HTTP (Figs. 9–10 of [1]). It shows much higher TNR, but its average TPR is "considerably low" (Table II of [1]).

■ TEAM'S OWN ANALYSIS / CONTRIBUTION

Do the results support the claims? Partly. The claim of competitiveness with offline training is supported on the one Kitsune-Mirai trace. However, that trace is 84% malicious and the attack begins after a long benign period. This is the most favourable setting for a self-labelling scheme, because the model is fully trained before the attack starts, and the attack is abrupt, so it is flagged at once and never learned. The claim that SSID enables adaptation to time-varying traffic is motivated but never directly tested: no experiment contains a controlled benign drift. The CDIS results show that much of the gain comes from higher TNR, while TPR is low (Table II of [1]), so the improvement is in false alarms rather than in detection.

6. Critical Assessment

■ TEAM'S OWN ANALYSIS / CONTRIBUTION

This section is entirely our own analysis. Where we say an issue was confirmed experimentally, the evidence is in §8.

6.1 Strengths

- Truly label-free and online: no offline dataset, no human labelling, and a cold start of one packet. Few self-supervised IDSs manage this (Table 1).
- Decides when to learn: trust-triggered updates replace a hand-picked retraining interval. In our runs, SSID's steady state performs very few updates compared with periodic retraining.
- Model-agnostic: the same controller works with AADRNN, MLP and CDIS. We confirmed this with an MLP backbone.
- Very lightweight: it uses 3–6 header metrics, O(1) statistics for C_{rep}, and OS-ELM updates of the output layer only. It is well suited to IoT gateways.
- Batch clearing on alarm (eq. 16) is a simple and effective defence against learning the packets just before an abrupt attack.

6.2 Restrictive assumptions

Assumption	Why it is restrictive	Consequence
First packet is benign; network clean at deployment	An IDS is often deployed because a network is suspected compromised.	If an infection is already present, it is learned as "normal" and never flagged.
TT and PL are exponential and independent	IoT packet lengths are strongly multimodal (ACK ≈ 60 B vs. MTU 1,500 B), and inter-arrival times are bursty or periodic.	C_{rep} compares only means. Two very different distributions with equal means give $C_{rep} = 1$.
Cumulative statistics	λ_l, λ_o and κ average over the whole history.	After months of operation, a drift barely moves the means, so C_{rep} reacts slowly.
"Observed normal" = packets the IDS judged normal	This uses the IDS's own decisions, the same quantity being trusted.	Circular: C_{rep} cannot reveal that the IDS is systematically wrong.
Fixed γ = 0.25	y is a reconstruction error, so its scale depends on the normalisation of the metrics.	γ has to be re-tuned for every metric set or normalisation, which contradicts the claim of no human intervention.
Data adequacy via W	W counts all parameters, but only W_H is updated online.	Δ overstates the data needed in the online phase.

Table 3 — Assumptions identified by the team.

6.3 Technical drawbacks and errors found in the paper

- Self-poisoning / boiling-frog vulnerability (confirmed experimentally). Training labels come from the IDS's own outputs. An attacker can raise the attack intensity slowly, so each batch is only slightly more anomalous than what has been learned. Each batch stays below γ, gets accepted and learned, and the model of "normal" follows the attack. Batch clearing (16) cannot help because the alarm never fires. This is the main threat to any self-training IDS, and the paper does not discuss it.
- Bootstrap deadlock (confirmed in implementation). At l = 0 we have Δ = 0 (fewer than W samples), so C_{gen} = ½κ = ¼(1 − E(0)). If the untrained IDS outputs y = 1 (an untrained detector reasonably reports "unknown" as maximum anomaly), then E(0) = 1, κ = 0, Γ = 0 and q = 1 forever: no packet is ever selected and learning never starts. Start-up only works because an untrained model outputs y < 1 by chance. A related problem: C_{rep} needs observed-normal statistics, but at start-up no packet is judged normal.

- κ is biased upward. The weights $(½)^{l-k+1}$ in (35) sum to $1 - 2^{-(l+1)} < 1$. With a constant error E, $\kappa = 1 - E(1 - 2^{-(l+1)})$. At l = 0, κ = 1 − E/2, which overstates knowledge exactly when the model knows least. We use the bias-corrected form $\kappa = 1 - \Sigma w_k E(k) / \Sigma w_k$.
- Typos and inconsistencies. (a) Eq. (38) reads $[1 - \min(\cdot) + \Sigma(½)^{l-k+1}E(k)]/2$. From (37) it must be $[2 - \min(\cdot) - \Sigma(½)^{l-k+1}E(k)]/2$. (b) Eq. (10) initialises O_0 with $\tilde{X}_H$ (the output), but (11)–(12) use the hidden output $\tilde{X}_{H-1}$. (c) Eq. (28) is labelled D^{PL} although it gives D^{TT}. (d) The window in (15) is j = i−l+1..i (l packets), but in (16) it is j = i−l..i (l+1 packets). (e) $O_0 = (X^T X)^{-1}$ is singular when fewer than M+1 samples have been learned, which is always the case at the start of l = 0. Regularisation is needed but not stated. (f) The discriminant in (4) is negative for small Λ, and no input domain is given.
- Malicious selection (p^+) is unused. An auto-associative IDS learns only benign data, so the p^+ branch only lowers the effective selection rate. The framework's symmetric formulation is not exploited.
- No bottleneck in the AAM. H = M layers of M units means the network can approach the identity map. Anomalies are then detected only through the saturation of ζ, and the online OS-ELM updates steadily make the network more permissive. This makes poisoning worse.
- Unbounded batch memory. While Γ ≥ Θ, nearly every packet is selected ($p^- \approx 0.95$) but no update happens, so B_l grows without limit. That is a problem on memory-constrained IoT nodes. We cap B at 5,000 packets.

6.4 Gaps in the evaluation

- Malicious-traffic detection is evaluated on one trace (Kitsune-Mirai) that is 84% malicious, so accuracy is inflated. There are no results over several runs, even though packet selection is random.
- No sensitivity analysis for Θ, γ, l and K, and no justification for γ = 0.25.
- Missing baseline: KitNET [17], an online unsupervised autoencoder IDS published with the same dataset. Also missing: streaming detectors such as Half-Space Trees or RRCF.
- The motivating case, benign concept drift, is never tested on its own, and neither are adversarial or gradual attacks.
- There are no runtime, memory or energy measurements, despite the IoT-lightweight claim.
- Label granularity: metrics are aggregated over 500 packets or 100 s, yet packets are scored with per-packet labels. Benign packets during an attack window inevitably look malicious. In our evaluation, FPR is measured on benign packets outside attack windows.

6.5 Unresolved issues

Several questions remain open. How should SSID tell benign drift from a slow attack, given that both look alike in these features? How should trust be calibrated against ground truth when a little labelled data exists? How can the AAM be kept from becoming more permissive after many online updates? And how should several SSID nodes share knowledge without sharing poisoned models (the federated extension the authors mention)?

7. Proposed Improvements: SSID+

■ TEAM'S OWN ANALYSIS / CONTRIBUTION

Each change targets a limitation from §6, can be switched on or off independently (ablation in §8.5), and keeps SSID's main property: no labels and no offline data.

#	Change	Limitation addressed	Technical justification	Expected benefit / challenge
1	Sliding-window Jensen–Shannon representativeness: $C_{rep} = 1 - ½[JS(TT) + JS(PL)]$ over 16-bin histograms of log TT and log PL, on the last 3,000 learned and observed packets	Exponential assumption; mean-only comparison; cumulative statistics	JS divergence in base 2 is symmetric and bounded in [0,1], so it needs no e^{-D} mapping. Histograms capture multimodality. A window keeps it responsive.	Notices shape changes and reacts faster to drift. Costs O(bins) per evaluation, and the bin ranges must be chosen.
2	Bias-corrected $\kappa = 1 - \Sigma w_k E(k) / \Sigma w_k$	Upward bias of (35)	This is a proper weighted mean, so $\kappa = 1 - E$ for a constant error.	Trust is no longer inflated early on. Bootstrapping is slightly slower.
3	Confidence-gated pseudo-labelling: accept as benign only if $y_i \le \min(\gamma/2, \mu_{acc} + 3\sigma_{acc})$	Self-poisoning; confirmation bias	This is the confident-learning principle used in self-training: learn only from high-confidence pseudo-labels. The bound follows the IDS's own accepted outputs.	Stops ambiguous packets from being learned. Too strict a gate slows adaptation to abrupt benign drift.
4	Adaptive intrusion threshold $\gamma_t = \text{clip}(\mu_{acc} + k\sigma_{acc}, 0.03, \gamma)$, $k = 6$	Scale-dependent fixed γ; slow ramp stays under 0.25	The alarm threshold is set relative to the IDS's own benign error distribution, like a control chart.	No tuning per metric set. An attack is flagged as soon as it leaves the benign error band, so its batches are cleared by (16).
5	Forgetting-factor OS-ELM ($\lambda_f = 0.995$)	Equal weight on all history slows adaptation	This is standard FOS-ELM / exponentially weighted RLS.	Faster tracking of drift. Used alone it would make poisoning easier, so it is paired with 3 and 4.

Table 4 — SSID+ components.

7.1 Further extensions (proposed, not implemented)

- Drift-versus-attack discrimination with a sequential change detector (Page–Hinkley or ADWIN) on each metric. Drift that keeps the shape of benign traffic (for example, packet-size mix) could be learned, while a shift in direction toward known attack signatures (tiny packets, high rate) is held back for confirmation.
- Anchor memory: keep a small reservoir of early, high-confidence benign samples and reject any update that worsens reconstruction on the anchors by more than a margin. This is a safeguard against catastrophic poisoning.
- Federated SSID (the authors' future work): nodes share Γ-weighted output-layer updates, with robust aggregation (median or trimmed mean) so that one poisoned node cannot spread its model.
- Bottleneck AAM (M → M−1 → M) or a KitNET-style ensemble, to stop the auto-associative memory from drifting toward the identity map.
- Semi-supervised calibration: when a few labelled packets exist, use them as the validation set that the paper's κ allows for but never uses.

8. Implementation and Experimental Validation

■ TEAM'S OWN ANALYSIS / CONTRIBUTION

8.1 Implementation

We implemented the full pipeline from scratch in Python (NumPy, scikit-learn, Matplotlib). The code is in 3_Source_Code/, and each line of the SSID loop is annotated with the matching equation number of [1].

File	Content
ssid/traffic.py	Scenario generator; real-packet CSV loader; causal extraction of the 3 metrics (vectorised, O(n)).
ssid/models.py	DRNN activation ζ (eq. 4), FISTA for non-negative L1 least squares (eq. 7), AADRNN with OS-ELM (eqs. 10–12) and an optional forgetting factor; M-M-M-M sigmoid MLP auto-encoder with Adam; decision maker (eq. 5).
ssid/framework.py	SSID controller (eqs. 13–38) with SSID+ switches; offline and incremental baselines.
run_experiments.py	All scenarios × seeds × methods, ablation, Θ×γ sensitivity, tables and figures.
pcap_to_csv.py	Converts Kitsune/Bot-IoT pcaps and labels into the CSV used by --csv (to run on the real datasets).

Implementation choices not specified in the paper (documented in the code): (i) a ridge term of 10^{-3} in O_0, because it is singular with few samples; (ii) the inputs to the hidden layers are min-max scaled with statistics frozen at the last initial fit, because with low-variance inputs the non-negative L1 problem (7) collapses onto the bias row and the hidden layers carry no information; (iii) the last hidden layer is standardised before the linear output layer; (iv) an untrained model outputs y = 0.5 (with y = 1 the bootstrap deadlocks, §6.3); (v) during l = 0 every packet counts as observed traffic for C_{rep}; (vi) during l = 0 the hidden layers are refitted at $|B_0| = 2^k$ and every 100 packets, with OS-ELM updates in between; (vii) the batch B is capped at 5,000; (viii) the first 500 packets are skipped so the 500-packet metric window is full.

8.2 Experimental setup

Data. Kitsune and Bot-IoT need registration and several GB of pcap, so we evaluate on a reproducible synthetic IoT trace generator built to recreate the conditions that matter for SSID. The trace lasts 3,000 s and has six benign devices with Poisson arrivals and bimodal packet lengths (control packets ≈ 66 B, data packets 90 B + an exponential component). The attack phase (last third) uses a Mirai-like flood at 60 packets/s of 54–80 B scan/SYN packets. There are three scenarios: mirai (stationary benign traffic and an abrupt attack, close to the paper's setting); mirai_drift (four new camera-like devices join one at a time in the middle third: benign concept drift); and slow_ramp (drift plus an attack whose rate ramps linearly from 0 to 60 packets/s over 900 s, an adversarial "boiling-frog" test). Each scenario has about 60–90 k packets and is run with 5 random seeds.

Baselines, following the paper's protocol as closely as possible: Offline-AADRNN, Offline-KNN (mean 5-NN distance) and Offline-IForest, all trained on the first 5,000 oracle-labelled benign packets; Incremental-AADRNN (first 750 packets assumed benign, then retrained every 750 packets on packets it judged benign); SSID-AADRNN and SSID-MLP with the paper's parameters (Θ = 0.95, l = 10, K = 100, γ = 0.25); and SSID+ (ours). Metrics: TPR on attack packets; TNR on benign packets outside attack windows (see §6.4); balanced accuracy, which handles class imbalance; threshold-free ROC-AUC; packets learned; and processing time per packet (Apple-silicon laptop, single thread, Python).

8.3 Results

Scenario mirai (abrupt attack, no drift)

Method	TPR	TNR	Balanced acc.	AUC	Packets learned	µs / packet
Offline-AADRNN	0.770 ± 0.386	1.0000 ± 0.0000	0.885 ± 0.193	0.9996	-	0.2
Incremental-AADRNN	0.000 ± 0.000	1.0000 ± 0.0000	0.500 ± 0.000	0.9701	-	5.0
Offline-KNN	1.000 ± 0.000	0.6881 ± 0.0396	0.844 ± 0.020	0.9999	-	1.1
Offline-IForest	0.799 ± 0.400	0.9770 ± 0.0173	0.888 ± 0.195	0.9991	-	1.8
SSID-AADRNN (paper)	0.990 ± 0.003	0.9998 ± 0.0000	0.995 ± 0.002	0.9997	519	4.7
SSID-MLP (paper)	0.992 ± 0.001	0.9998 ± 0.0000	0.996 ± 0.001	0.9996	525	13.2
SSID+ (ours)	0.995 ± 0.001	0.9998 ± 0.0000	0.997 ± 0.000	0.9997	1,505	43.7

Table 5 — Mean ± std over 5 seeds.

Scenario mirai_drift (gradual benign drift, then abrupt attack)

Method	TPR	TNR	Balanced acc.	AUC	Packets learned	µs / packet
Offline-AADRNN	0.773 ± 0.387	1.0000 ± 0.0000	0.887 ± 0.193	0.9993	-	0.1
Incremental-AADRNN	0.002 ± 0.003	1.0000 ± 0.0000	0.501 ± 0.001	0.9092	-	4.6
Offline-KNN	1.000 ± 0.000	0.3515 ± 0.0197	0.676 ± 0.010	0.9996	-	1.5
Offline-IForest	0.800 ± 0.398	0.3953 ± 0.0106	0.598 ± 0.197	0.7556	-	1.6
SSID-AADRNN (paper)	0.969 ± 0.015	0.9999 ± 0.0000	0.984 ± 0.007	0.9991	5,519	15.1
SSID-MLP (paper)	0.981 ± 0.009	0.9999 ± 0.0000	0.990 ± 0.005	0.9988	5,525	13.0
SSID+ (ours)	0.996 ± 0.001	0.9999 ± 0.0000	0.998 ± 0.000	0.9996	5,505	90.7

Table 6 — Mean ± std over 5 seeds.

Scenario slow_ramp (gradual drift, then slowly ramping attack)

Method	TPR	TNR	Balanced acc.	AUC	Packets learned	µs / packet
Offline-AADRNN	0.512 ± 0.266	1.0000 ± 0.0000	0.756 ± 0.133	0.9984	-	0.2
Incremental-AADRNN	0.000 ± 0.000	1.0000 ± 0.0000	0.500 ± 0.000	0.9993	-	4.7
Offline-KNN	1.000 ± 0.000	0.3515 ± 0.0197	0.676 ± 0.010	0.9994	-	1.6
Offline-IForest	0.793 ± 0.387	0.3953 ± 0.0106	0.594 ± 0.191	0.7491	-	2.2
SSID-AADRNN (paper)	0.002 ± 0.004	0.9999 ± 0.0000	0.501 ± 0.002	0.8968	18,864	44.2
SSID-MLP (paper)	0.003 ± 0.005	0.9999 ± 0.0000	0.501 ± 0.003	0.6571	15,084	18.7
SSID+ (ours)	0.985 ± 0.004	0.9999 ± 0.0000	0.992 ± 0.002	0.9994	5,505	103.9

Table 7 — Mean ± std over 5 seeds.

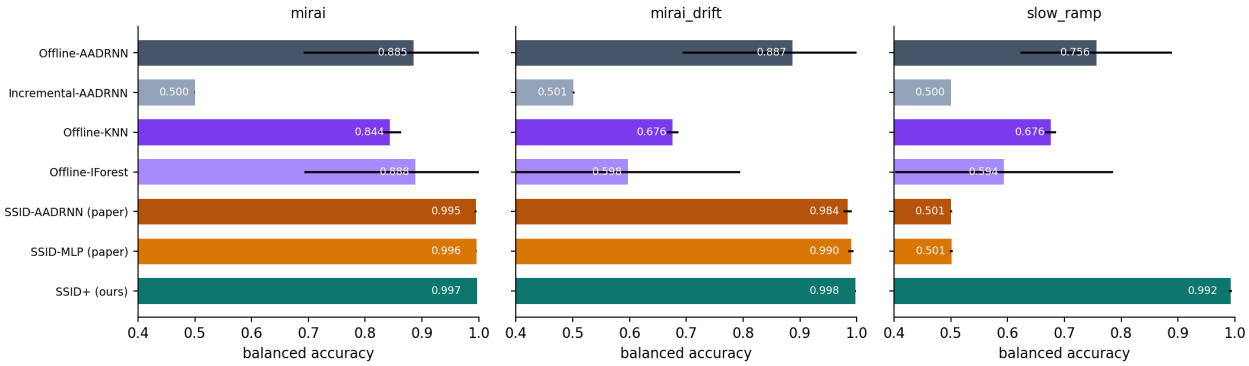


Figure 3 — Balanced accuracy (mean ± std over 5 seeds) for all methods and scenarios.

8.4 Discussion

- The paper's claims reproduce in its own setting. On mirai, SSID-AADRNN reaches balanced accuracy 0.995 and AUC 0.9997, after learning from only about 519 packets with no labels. This matches the offline model's AUC, which used 5,000 oracle-labelled packets. Incremental learning collapses (TPR ≈ 0) because it retrains on attack windows. This agrees with the paper's Fig. 6.
- A fixed γ is fragile for offline models. Offline-AADRNN has high AUC but very variable TPR (0.77 ± 0.39) at γ = 0.25. The ranking is good; the threshold is not. This supports our point that γ depends on scale.
- Benign drift. The offline KNN and IForest detectors, which cannot adapt, flag almost all post-drift benign traffic (TNR ≈ 0.35–0.40). SSID tracks the drift with zero false positives in the drift phase, which confirms the paper's main motivation. That motivation was not tested in [1]. SSID+ also improves TPR from 0.969 to 0.996.
- Self-poisoning. In slow_ramp, both of the paper's SSID variants learn the attack as normal: TPR is 0.002 (AADRNN) and 0.003 (MLP), and the learned set grows to about 18,864 packets. Fig. 4 shows the mechanism: the IDS output climbs with the attack but stays just below γ, Γ dips, an update is triggered, and the output collapses back near 0 while the attack is still running. SSID+ keeps detecting the attack (TPR 0.985 ± 0.004) with unchanged TNR, because its adaptive threshold (≈0.09 here) fires early, so the attack batches are cleared by (16) and never learned.
- Cost. SSID+ is roughly 3–6× slower per packet in our Python code, mostly because of the JS histograms. At about 40–100 µs per packet it can still handle more than 10,000 packets/s on one core, and a C implementation would remove most of the overhead.

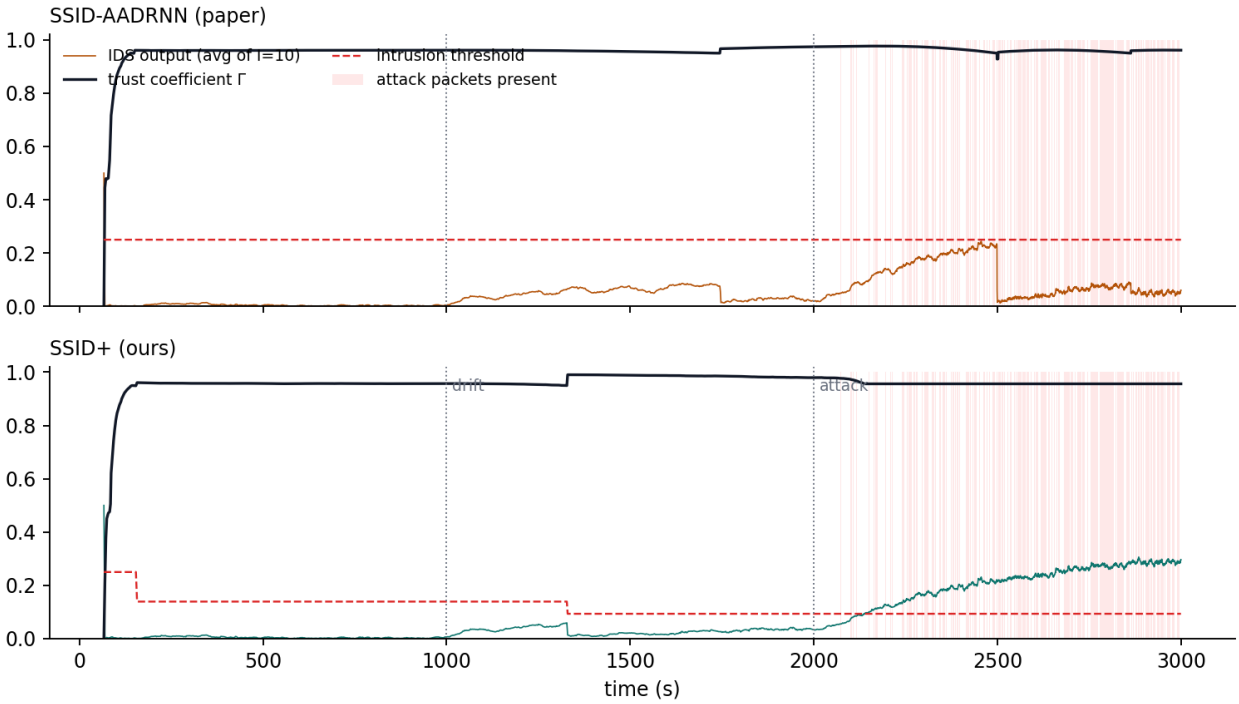


Figure 4 — Slow-ramp scenario, seed 0: IDS output, trust Γ and intrusion threshold over time. Top: the paper's SSID absorbs the attack at t ≈ 2,500 s. Bottom: SSID+ keeps the attack above its adaptive threshold (style follows Fig. 5 of [1]).

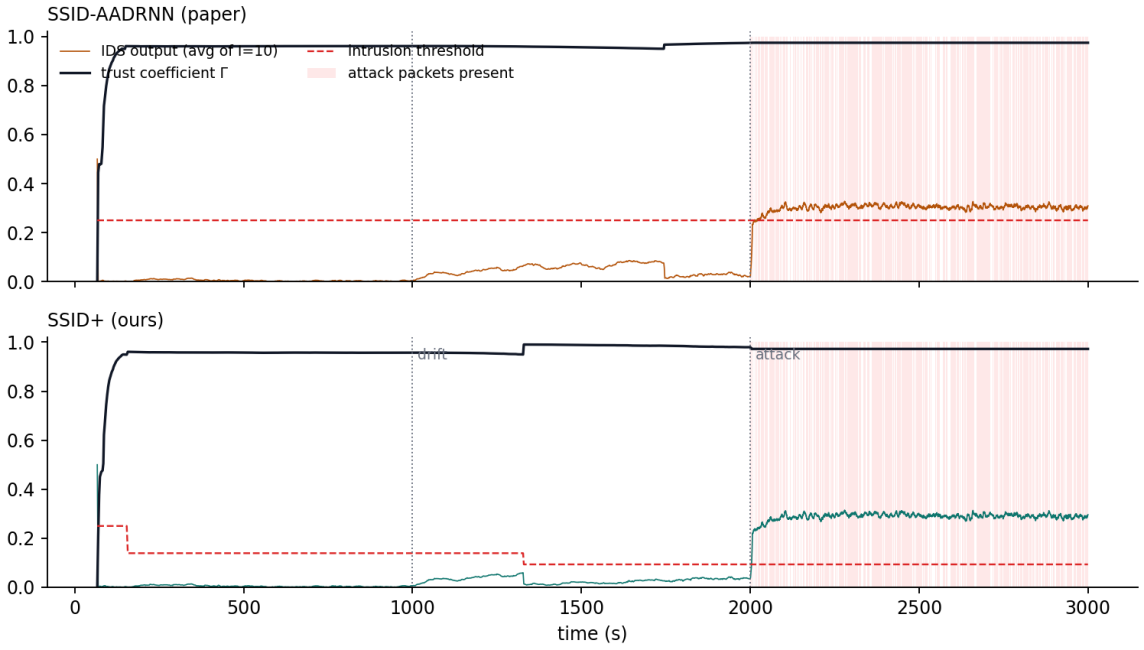


Figure 5 — Drift scenario: both frameworks follow the benign drift (new devices at 1,000–1,600 s) without false alarms.

8.5 Ablation study

Each SSID+ component was added on its own to the paper's SSID (slow_ramp scenario, 5 seeds):

Variant	TPR	Balanced acc.	Packets learned
SSID-AADRNN (paper)	0.002 ± 0.004	0.501 ± 0.002	18,864
SSID + gate	0.345 ± 0.304	0.672 ± 0.152	8,937
SSID + adaptive-gamma	0.957 ± 0.023	0.978 ± 0.012	5,519
SSID + JS-rep	0.000 ± 0.000	0.500 ± 0.000	34,148
SSID + unbiased-kappa	0.161 ± 0.317	0.580 ± 0.159	21,190
SSID + forgetting	0.002 ± 0.004	0.501 ± 0.002	16,094
SSID+ (ours)	0.985 ± 0.004	0.992 ± 0.002	5,505

Table 8 — Ablation on slow_ramp.

The adaptive threshold contributes most on its own (TPR 0.96). The confidence gate helps partly (0.34, with high variance). JS representativeness and forgetting do not resist poisoning by themselves. They exist to make adaptation faster, and alone they even increase the amount learned. The full combination gives the highest TPR with the lowest variance and learns about 3× fewer packets, so gating and the adaptive threshold cover each other's weaknesses. In short, poisoning resistance comes from what gets learned (gate, γ_t), and the other components only change how fast learning happens.

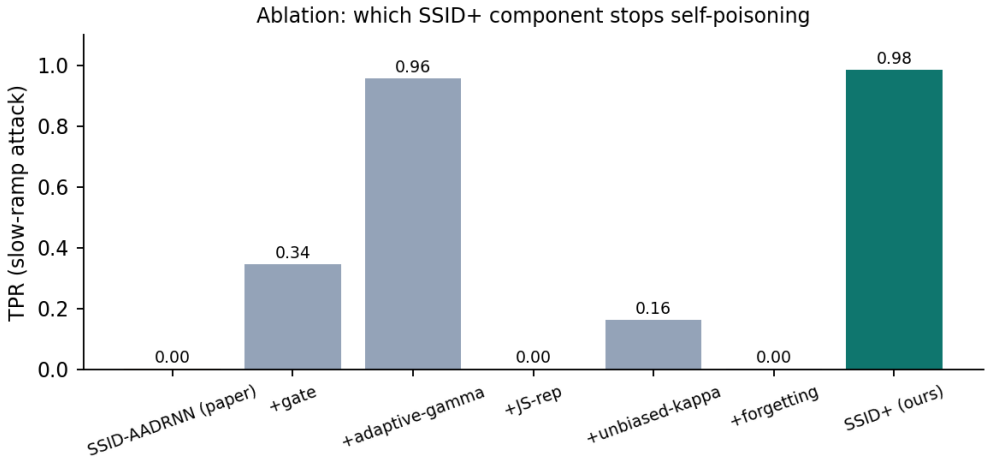


Figure 6 — Ablation: TPR on the slow-ramp attack.

8.6 Hyper-parameter sensitivity of the paper's SSID

Over $\Theta \in \{0.90, 0.95, 0.98\} \times \gamma \in \{0.10, 0.25, 0.40\}$ on mirai_drift (3 seeds), the balanced accuracy of SSID ranges from 0.500 to 0.998. $\gamma = 0.40$ breaks detection completely (≈0.5), and moving Θ away from 0.95 costs about 10 points. The paper's choice works well here, but the result depends strongly on parameters the paper presents as fixed. This supports both the "no human intervention" critique and the adaptive-γ proposal.

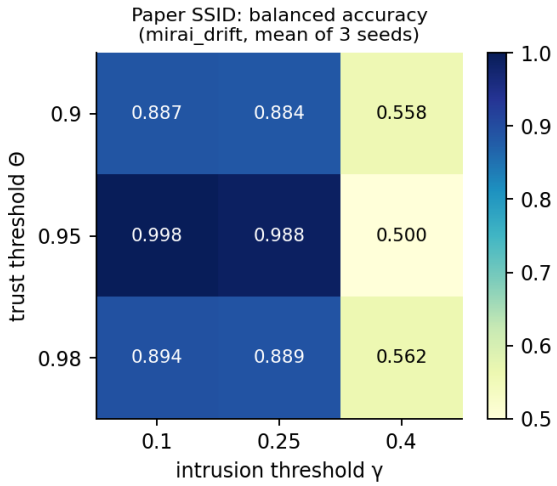


Figure 7 — Sensitivity of the paper's SSID to Θ and γ.

8.7 Threats to validity

Our numbers come from a synthetic generator, not from Kitsune or Bot-IoT, so they show mechanisms (adaptation, poisoning) rather than absolute performance on those datasets. The metric normalisation (log ranges) is our own, and γ depends on it. The AADRNN needed conditioning choices (§8.1) that may differ from the authors' code. The slow-ramp attack is deliberately adversarial. We provide pcap_to_csv.py so the exact same pipeline can be run on the real Kitsune-Mirai capture.

9. Conclusion

■ Authors: SSID shows that a deep anomaly-based IDS can train itself fully online from its own trust-weighted decisions. It needs no offline data or labels, adapts to traffic changes, and is competitive with offline training on Kitsune-Mirai and Bot-IoT.

■ Team: Our re-implementation confirms these claims for abrupt attacks and for gradual benign drift, which the paper motivates but does not test. We also show that the self-labelling loop has a serious blind spot: a slowly ramping attack is learned as normal (TPR ≈ 0). We found a bootstrap deadlock, an upward bias in κ, several equation typos, restrictive traffic assumptions and gaps in the evaluation. Our SSID+ variant combines distribution-free windowed representativeness, a bias-corrected κ, confidence-gated pseudo-labels, an adaptive threshold and a forgetting factor. It closes the poisoning gap (TPR 0.985) without losing performance elsewhere. The remaining open problem is telling abrupt benign drift from an attack without labels. We see change detection, anchor memories and robust federated aggregation as the next steps.

References

[1] M. Nakıp and E. Gelenbe, “Online self-supervised deep learning for intrusion detection systems,” IEEE Trans. Inf. Forensics Security, vol. 19, pp. 5668–5683, 2024.

[7] M. Nakıp and E. Gelenbe, “Botnet attack detection with incremental online learning,” in Security in Computer and Information Sciences, Springer, 2022.

[8] E. Gelenbe and M. Nakıp, “Traffic based sequential learning during botnet attacks to identify compromised IoT devices,” IEEE Access, vol. 10, 2022.

[17] Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” NDSS, 2018.

[18] N. Koroniotis et al., “Towards the development of realistic botnet dataset in the Internet of Things: Bot-IoT,” FGCS, vol. 100, 2019.

[48] E. Gelenbe and Y. Yin, “Deep learning with random neural networks,” IJCNN, 2016.

[58] A. Beck and M. Teboulle, “A fast iterative shrinkage-thresholding algorithm for linear inverse problems,” SIAM J. Imaging Sci., 2009.

[59] N.-Y. Liang et al., “A fast and accurate online sequential learning algorithm for feedforward networks,” IEEE Trans. Neural Netw., 2006.

[T1] J. Lin, “Divergence measures based on the Shannon entropy,” IEEE Trans. Inf. Theory, vol. 37, no. 1, 1991. (JS divergence, SSID+)

[T2] B. Biggio and F. Roli, “Wild patterns: Ten years after the rise of adversarial machine learning,” Pattern Recognition, vol. 84, 2018. (poisoning)

[T3] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing (ADWIN),” SDM, 2007. (drift detection)

[T4] J.-S. Lim, “Partitioned online sequential extreme learning machine / forgetting-factor OS-ELM,” Neurocomputing, 2013. (forgetting OS-ELM)

[T5] E. Arazo et al., “Pseudo-labeling and confirmation bias in deep semi-supervised learning,” IJCNN, 2020. (confidence gating)

Reference numbers [1]–[66] follow the analysed paper; [T1]–[T5] are additional references used by the team. Figures 1–2 are the team's re-drawings of figures in [1]. All result figures and tables in §8 are the team's own.

Appendix A — Reproducing the results

```
cd 3_Source_Code
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python make_diagrams.py                 # architecture figures
python run_experiments.py --quick       # 1 seed  (~3 min)
python run_experiments.py               # 5 seeds + ablation + sensitivity (~15 min)
# real data (optional):
tshark -r Mirai_pcap.pcap -T fields -e frame.time_epoch -e frame.len -E separator=, > f.csv
python pcap_to_csv.py f.csv Mirai_labels.csv kitsune.csv && python run_experiments.py --csv kitsune.csv
```

Appendix B — Individual contributions

Member	Contribution (fill in)
<Member 1>	§1–2, slides 1–6, video part A
<Member 2>	§3–4, Algorithm 1, slides 7–12, video part B
<Member 3>	§5–6, error analysis of equations, slides 13–19, video part C
<Member 4>	§7–8, code, experiments, slides 20–27, video part D